AuthXDependency

class authx.dependencies.AuthXDependency

```
AuthXDependency(_from, request, response)
```

Bases: `Generic[T]`

A dependency class for managing authentication-related operations within a request-response context.

Provides a convenient interface for creating tokens, setting and unsetting cookies, and retrieving the current authenticated subject.

ATTRIBUTE	DESCRIPTION
`request`	The HTTP request associated with the current authentication context. **TYPE:** `Request`
`response`	The HTTP response associated with the current authentication context. **TYPE:** `Response`

METHOD	DESCRIPTION
`create_access_token`	Generate an access token for a given user.
`create_refresh_token`	Generate a refresh token for a given user.
`set_access_cookies`	Set access token cookies in the response.
`set_refresh_cookies`	Set refresh token cookies in the response.
`unset_access_cookies`	Remove access token cookies from the response.
`unset_refresh_cookies`	Remove refresh token cookies from the response.
`unset_cookies`	Remove all authentication-related cookies from the response.
`get_current_subject`	Asynchronously retrieve the currently authenticated subject.

Initialize the authentication dependency with request, response, and security context.

Sets up the core components required for managing authentication operations within the current request-response cycle.

PARAMETER	DESCRIPTION
`_from`	The AuthX instance managing authentication mechanisms. **TYPE:** `AuthX[T]`
`request`	The incoming HTTP request object. **TYPE:** `Request`
`response`	The HTTP response object to be potentially modified during authentication. **TYPE:** `Response`

Source code in `authx/dependencies.py`

```python
def __init__(
    self,
    _from: "AuthX[T]",
    request: Request,
    response: Response,
) -> None:
    """Initialize the authentication dependency with request, response, and security context.

    Sets up the core components required for managing authentication operations within the current
request-response cycle.

    Args:
        _from: The AuthX instance managing authentication mechanisms.
        request: The incoming HTTP request object.
        response: The HTTP response object to be potentially modified during authentication.
    """
    self._response = response
    self._request = request
    self._security = _from
```

`attr` request `property`

```
request
```

Retrieve the HTTP request associated with the current authentication context.

Provides read-only access to the request object used during authentication processing.

RETURNS	DESCRIPTION
`Request`	The HTTP request object stored in the authentication dependency.

`attr` response `property`

```
response
```

Retrieve the HTTP response associated with the current authentication context.

Provides read-only access to the response object used during authentication processing.

RETURNS	DESCRIPTION
`Response`	The HTTP response object stored in the authentication dependency.

`meth` create_access_token

```
create_access_token(uid, fresh=False, headers=None, expiry=None, data=None, audience=None, *args,
**kwargs)
```

Generate an access token for a specific user with customizable parameters.

Delegates token creation to the underlying security mechanism with flexible configuration options.

Args: uid: Unique identifier of the user for whom the token is being created. fresh: Flag indicating whether the token should be marked as a fresh authentication. headers: Optional custom headers to include in the token. expiry: Optional expiration time for the token. data: Optional additional data to be encoded in the token. audience: Optional target audience for the token. *args: Variable positional arguments for additional flexibility.* *kwargs: Variable keyword arguments for additional configuration.

Returns: A string representing the generated access token.

Source code in `authx/dependencies.py`

```
def create_access_token(
    self,
    uid: str,
    fresh: bool = False,
    headers: Optional[dict[str, Any]] = None,
    expiry: Optional[DateTimeExpression] = None,
    data: Optional[dict[str, Any]] = None,
    audience: Optional[StringOrSequence] = None,
    *args: Any,
    **kwargs: Any,
) -> str:
    """Generate an access token for a specific user with customizable parameters.

    Delegates token creation to the underlying security mechanism with flexible configuration
options.

    Args:
    uid: Unique identifier of the user for whom the token is being created.
    fresh: Flag indicating whether the token should be marked as a fresh authentication.
    headers: Optional custom headers to include in the token.
    expiry: Optional expiration time for the token.
    data: Optional additional data to be encoded in the token.
    audience: Optional target audience for the token.
    *args: Variable positional arguments for additional flexibility.
    **kwargs: Variable keyword arguments for additional configuration.

    Returns:
    A string representing the generated access token.
    """
    return self._security.create_access_token(uid, fresh, headers, expiry, data, audience, *args,
**kwargs)
```

`meth` create_refresh_token

```
create_refresh_token(uid, headers=None, expiry=None, data=None, audience=None, *args, **kwargs)
```

Generate a refresh token for a specific user with customizable parameters.

Delegates refresh token creation to the underlying security mechanism with flexible configuration options.

Args: uid: Unique identifier of the user for whom the refresh token is being created. headers: Optional custom headers to include in the token. expiry: Optional expiration time for the token. data: Optional additional data to be encoded in the token. audience: Optional target audience for the token. *args: Variable positional arguments for additional flexibility.* *kwargs: Variable keyword arguments for additional configuration.

Returns: A string representing the generated refresh token.

Source code in `authx/dependencies.py`

```
def create_refresh_token(
    self,
    uid: str,
    headers: Optional[dict[str, Any]] = None,
    expiry: Optional[DateTimeExpression] = None,
    data: Optional[dict[str, Any]] = None,
    audience: Optional[StringOrSequence] = None,
    *args: Any,
    **kwargs: Any,
) -> str:
    """Generate a refresh token for a specific user with customizable parameters.

    Delegates refresh token creation to the underlying security mechanism with flexible configuration
options.

    Args:
    uid: Unique identifier of the user for whom the refresh token is being created.
    headers: Optional custom headers to include in the token.
    expiry: Optional expiration time for the token.
    data: Optional additional data to be encoded in the token.
    audience: Optional target audience for the token.
    *args: Variable positional arguments for additional flexibility.
    **kwargs: Variable keyword arguments for additional configuration.

    Returns:
    A string representing the generated refresh token.
    """
    return self._security.create_refresh_token(uid, headers, expiry, data, audience, *args, **kwargs)
```

`meth` create_token_pair

```
create_token_pair(uid, fresh=False, headers=None, access_expiry=None, refresh_expiry=None,
data=None, audience=None, access_scopes=None, refresh_scopes=None)
```

Generate an access and refresh token pair.

Delegates to the underlying AuthX instance to create both tokens.

PARAMETER	DESCRIPTION
`uid`	Unique identifier of the user. **TYPE:** `str`
`fresh`	Whether the access token should be marked as fresh. **TYPE:** `bool` **DEFAULT:** `False`
`headers`	Optional custom JWT headers. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`
`access_expiry`	Optional access token expiry override.

PARAMETER	DESCRIPTION
	TYPE: `Optional[DateTimeExpression]` **DEFAULT:** `None`
`refresh_expiry`	Optional refresh token expiry override. **TYPE:** `Optional[DateTimeExpression]` **DEFAULT:** `None`
`data`	Optional additional data for both tokens. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`
`audience`	Optional audience claim. **TYPE:** `Optional[StringOrSequence]` **DEFAULT:** `None`
`access_scopes`	Optional scopes for the access token. **TYPE:** `Optional[list[str]]` **DEFAULT:** `None`
`refresh_scopes`	Optional scopes for the refresh token. **TYPE:** `Optional[list[str]]` **DEFAULT:** `None`

RETURNS	DESCRIPTION
`TokenResponse`	TokenResponse with access_token, refresh_token, and token_type.

Source code in `authx/dependencies.py`

```python
def create_token_pair(
    self,
    uid: str,
    fresh: bool = False,
    headers: Optional[dict[str, Any]] = None,
    access_expiry: Optional[DateTimeExpression] = None,
    refresh_expiry: Optional[DateTimeExpression] = None,
    data: Optional[dict[str, Any]] = None,
    audience: Optional[StringOrSequence] = None,
    access_scopes: Optional[list[str]] = None,
    refresh_scopes: Optional[list[str]] = None,
) -> TokenResponse:
    """Generate an access and refresh token pair.

    Delegates to the underlying AuthX instance to create both tokens.

    Args:
        uid: Unique identifier of the user.
        fresh: Whether the access token should be marked as fresh.
        headers: Optional custom JWT headers.
        access_expiry: Optional access token expiry override.
        refresh_expiry: Optional refresh token expiry override.
        data: Optional additional data for both tokens.
        audience: Optional audience claim.
        access_scopes: Optional scopes for the access token.
        refresh_scopes: Optional scopes for the refresh token.

    Returns:
        TokenResponse with access_token, refresh_token, and token_type.
    """
    return self._security.create_token_pair(
        uid=uid,
        fresh=fresh,
        headers=headers,
        access_expiry=access_expiry,
        refresh_expiry=refresh_expiry,
        data=data,
        audience=audience,
        access_scopes=access_scopes,
        refresh_scopes=refresh_scopes,
    )
```

`meth` set_access_cookies

```python
set_access_cookies(token, response=None, max_age=None)
```

Set access token cookies in the HTTP response.

Configures the response with access token cookies, using the provided token and optional parameters.

Args: token: The access token to be set as a cookie. response: Optional HTTP response object to set cookies on. Defaults to the stored response if not provided. max_age: Optional maximum age for the cookie before expiration.

Returns: None

Source code in `authx/dependencies.py`

```
def set_access_cookies(
    self,
    token: str,
    response: Optional[Response] = None,
    max_age: Optional[int] = None,
) -> None:
    """Set access token cookies in the HTTP response.

    Configures the response with access token cookies, using the provided token and optional
parameters.

    Args:
    token: The access token to be set as a cookie.
    response: Optional HTTP response object to set cookies on. Defaults to the stored response if not
provided.
    max_age: Optional maximum age for the cookie before expiration.

    Returns:
    None
    """
    self._security.set_access_cookies(token=token, response=(response or self._response),
max_age=max_age)
```

`meth` set_refresh_cookies

```
set_refresh_cookies(token, response=None, max_age=None)
```

Set refresh token cookies in the HTTP response.

Configures the response with refresh token cookies, using the provided token and optional parameters.

Args: token: The refresh token to be set as a cookie. response: Optional HTTP response object to set cookies on. Defaults to the stored response if not provided. max_age: Optional maximum age for the cookie before expiration.

Returns: None

Source code in `authx/dependencies.py`

```
def set_refresh_cookies(
    self,
    token: str,
    response: Optional[Response] = None,
    max_age: Optional[int] = None,
) -> None:
    """Set refresh token cookies in the HTTP response.

    Configures the response with refresh token cookies, using the provided token and optional
parameters.

    Args:
    token: The refresh token to be set as a cookie.
    response: Optional HTTP response object to set cookies on. Defaults to the stored response if not
provided.
    max_age: Optional maximum age for the cookie before expiration.

    Returns:
    None
    """
    self._security.set_refresh_cookies(token=token, response=(response or self._response),
max_age=max_age)
```

`meth` unset_access_cookies

```
unset_access_cookies(response=None)
```

Remove access token cookies from the HTTP response.

Clears the access token cookies from the specified or default response object.

Args: response: Optional HTTP response object to remove cookies from. Defaults to the stored response if not provided.

Returns: None

Source code in `authx/dependencies.py`

```
def unset_access_cookies(self, response: Optional[Response] = None) -> None:
    """Remove access token cookies from the HTTP response.

    Clears the access token cookies from the specified or default response object.

    Args:
    response: Optional HTTP response object to remove cookies from. Defaults to the stored response
if not provided.

    Returns:
    None
    """
    self._security.unset_access_cookies(response=(response or self._response))
```

`meth` unset_refresh_cookies

```
unset_refresh_cookies(response=None)
```

Remove refresh token cookies from the HTTP response.

Clears the refresh token cookies from the specified or default response object.

Args: response: Optional HTTP response object to remove cookies from. Defaults to the stored response if not provided.

Returns: None

Source code in `authx/dependencies.py`

```
def unset_refresh_cookies(self, response: Optional[Response] = None) -> None:
    """Remove refresh token cookies from the HTTP response.

    Clears the refresh token cookies from the specified or default response object.

    Args:
    response: Optional HTTP response object to remove cookies from. Defaults to the stored response
if not provided.

    Returns:
    None
    """
    self._security.unset_refresh_cookies(response=(response or self._response))
```

`meth` unset_cookies

```
unset_cookies(response=None)
```

Remove all authentication-related cookies from the HTTP response.

Clears both access and refresh token cookies from the specified or default response object.

Args: response: Optional HTTP response object to remove cookies from. Defaults to the stored response if not provided.

Returns: None

Source code in `authx/dependencies.py`

```python
def unset_cookies(self, response: Optional[Response] = None) -> None:
    """Remove all authentication-related cookies from the HTTP response.

    Clears both access and refresh token cookies from the specified or default response object.

    Args:
    response: Optional HTTP response object to remove cookies from. Defaults to the stored response
if not provided.

    Returns:
    None
    """
    self._security.unset_cookies(response=(response or self._response))
```

meth get_current_subject async

```python
get_current_subject()
```

Retrieve the currently authenticated subject from the request.

Asynchronously fetches the authenticated user or subject based on the current request context.

Returns: The authenticated subject if present, otherwise None.

Source code in `authx/dependencies.py`

```python
async def get_current_subject(self) -> Optional[T]:
    """Retrieve the currently authenticated subject from the request.

    Asynchronously fetches the authenticated user or subject based on the current request context.

    Returns:
    The authenticated subject if present, otherwise None.
    """
    return await self._security.get_current_subject(request=self._request)
```

March 20, 2026 March 19, 2026 GitHub